



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Jagoda WIĘCEK

Nr albumu: 300503

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr inż. Ewa Lach

KATEDRA Grafiki, Wizji Komputerowej i Systemów Cyfrowych

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2024

Tytuł pracy

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Streszczenie

(Streszczenie pracy.)

Słowa kluczowe

Model, agent, unity, uczenie maszynowe

Thesis title

2.5D board game with machine learning for computer players

Abstract

(Thesis abstract.)

Key words

Model, agent, unity, machine learning

Spis treści

1	Wstęp	1
1.1	Cel pracy	1
1.2	Zakres pracy	1
1.3	Charakterystyka rozdziałów	1
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Znane rozwiązania	4
3	Wymagania i narzędzia	5
3.1	Wymagania funkcjonalne	5
3.2	Wymagania niefunkcjonalne	6
3.3	Opis narzędzi	7
3.3.1	Unity	7
3.3.2	C#	7
3.3.3	Python	7
3.3.4	ML-Agents	8
3.4	Przypadki użycia	9
4	Specyfikacja zewnętrzna	11
4.1	Wymagania sprzętowe i programowe	11
4.2	Instalacja	11
4.2.1	Unity	11
4.2.2	Python	12
4.2.3	Biblioteka ml-agents	12
4.3	Aktywacja	13
4.4	Sposób obsługi	14
4.4.1	Zasady gry	14
4.4.2	Ekran aplikacji	15
4.4.3	Przebieg gry	17

5	Specyfikacja wewnętrzna	19
5.1	Biblioteka ml-agents	19
5.2	Klasa PlayerIA	19
5.3	Klasa GameController	19
5.4	Klasa Tile	19
6	Weryfikacja i walidacja	21
6.1	Testowanie	21
6.2	Wykryte błędy	21
7	Podsumowanie i wnioski	23
7.1	Napotkane problemy	23
7.2	Możliwy rozwój	23
	Bibliografia	25
	Spis skrótów i symboli	29
	Źródła	31
	Lista dodatkowych plików, uzupełniających tekst pracy	33
	Spis rysunków	35
	Spis tabel	37

Rozdział 1

Wstęp

Gry planszowe są formą rozrywki dla ludzi od wieków jak i formą intelektualnego wyzwania. We współczesnych czasach znaczna część życia podlega cyfryzacji, od komunikacji do płatności bankowych. Z tego powodu również wszelkie gry planszowe są przenoszone do cyfrowego świata. Takie rozwiązanie pozwala na rozpoczęcie rozgrywki ulubionej gry planszowej w świecie online wraz z graczami z całego świata bez żadnych ograniczeń.

1.1 Cel pracy

Celem pracy jest zaimplementowanie gry planszowej 2.5D w unity. Dodatkowo w ramach gry zostanie zaimplementowany gracz komputerowy z wykorzystaniem uczenia maszynowego.

1.2 Zakres pracy

- Przegląd algorytmów uczenia maszynowego dostępnego w unity.
- Zaimplementowanie graczy komputerowych z algorytmem uczenia maszynowego oraz graczy komputerowych z predefiniowanymi ruchami.
- Zaimplementowanie gry planszowej Rummikub 2.5D w unity.
- Zaprojektowanie mapy do gry.

1.3 Charakterystyka rozdziałów

- **Wstęp** - przedstawienie celu oraz zakres pracy, określenie grup docelowych
- **Analiza tematu** - zawiera ogólną analizę wszystkich możliwych rozwiązań tego projektu.

- **Wymagania i narzędzia** - przedstawienie narzędzi, wykorzystanych przy tworzeniu projektu, wymagania funkcjonalne, wymagania нефункционалне oraz diagram UML przedstawiający przypadki użycia.
- **Specyfikacja zewnętrzna** - Przetawia minimalne wymagania sprzętowe, jakie użytkownik powinien spełniać, aby korzystać z aplikacji. Zawiera również instrukcje dotyczące instalacji, aktywacji, zasady gry oraz ogólny opis użytkowania aplikacji.
- **Specyfikacja wewnętrzna** -
- **Weryfikacja i walidacja** -
- **Podsumowanie i wnioski** -

Rozdział 2

Analiza tematu

Tematem pracy jest cyfrowa gra planszowa z wykorzystaniem algorytmów sztucznej inteligencji. Kluczowym problemem jest zaimplementowanie inteligentnego gracza, który poprzez system prób i błędów, nauczy się grać. Wybranie gry planszowej ma duże znaczenie, im bardziej skomplikowane zasady gry, tym trudniej będzie modelowi nauczyć się zasad. Do realizacji tego projektu została wybrana gra planszowa o nazwie Rummikub. Rummikub to gra planszowa, w której gracze rywalizują, układając płytki w:

- grupy - te same liczby w różnych kolorach
- serie - kolejne liczby w tym samym kolorze.

Celem graczy jest jako pierwszy wyłożyć wszystkie płytki, przy czym pierwszy ruch wymaga uzyskania układów o sumie co najmniej 30 punktów. Gra kończy się, gdy gracz wyłoży wszystkie płytki, a punkty zależą od liczby pozostałych płytek u przeciwników.

2.1 Sformułowanie problemu

Cyfrowe wersje gier planszowych często są projektowane z myślą o rozgrywkach online z innymi graczami za pośrednictwem internetu. Jednak stworzenie gry z komputerowym przeciwnikiem opartym na algorytmach nauczania maszynowego wymaga odmiennego podejścia i zastosowania innych technik programistycznych, model nie powinien się uczyć na graniu z żywymi graczami, gdyż zajęło by mu to za długo. Algorytmy AI potrzebują dużej ilości powtórzeń aby nauczyć się zasad gry. W projekcie zostanie uwzględniony jeden żywy gracz, gracze komputerowi z predefiniowanymi ruchami oraz komputerowy gracz z algorytmem nauczania maszynowego.

2.2 Znane rozwiązania

Na rynku jest wiele gier planszowych, najpopularniejsze z nich doczekały się swojej cyfrowej wersji. Są nimi między innymi:

- Monopoli
- Cantan
- Rummikub - Wydana gra online Rummikub jest nastawiona na grę sieciową do komunikacji innych graczy z całego świata. Gra jest zaimplementowana w grafice 2D bez algorytmów sztucznej inteligencji.

Rozdział 3

Wymagania i narzędzia

W tej sekcji przedstawiono wymagania funkcjonalne i нефункционаłne projektu oraz narzędzia technologiczne i programistyczne, które zostały wykorzystane do jego realizacji. Opisane zostaną także kluczowe zasoby niezbędne do wdrożenia rozwiązania, takie jak środowiska programistyczne, biblioteki, frameworki oraz sprzęt.

3.1 Wymagania funkcjonalne

Interakcja z mapą :

- Gra musi pozwolić graczowi i agentowi na wybór płytki, którą chcą postawić bądź wejść z nią w interakcje.
- Gracz oraz agent powinni być w stanie postawić nową płytkę na mapie, przesunąć istniejącą bądź usunąć ją z mapy, jeśli jest to możliwe.
- Agent oraz gracz muszą wyłożyć serie bądź grupy płytek jak najszybciej.
- Gra musi ograniczać graczy, aby nie mogli tworzyć serii niedozwolonych w zasadach gry.

Ruchy gracza :

- Użytkownik musi mieć możliwość widzieć swoje płytki, wykladać je na mapę oraz tworzyć serie bądź grupy
- W trakcie tury gra na bieżąco weryfikuje czy daną płytkę można postawić, pod koniec tury sprawdza czy każdy wyłożony ciąg płytek posiada co najmniej 3 płytki w jednym ciągu.
- Gracz powinien móc zakończyć turę, cofnąć wszystkie akcje oraz dobrać nową płytkę.
- Użytkownik powinien móc posortować swoje płytki.

Komputerowi gracze :

- Jeśli jest to pierwsza tura gracza:
 - Sprawdza możliwość wyłożenia serii, gdzie jej suma wynosi co najmniej 30 lub więcej.
 - Sprawdza możliwość wyłożenia grupy, gdzie jej suma wynosi co najmniej 30 lub więcej.
- Gdy pierwsza tura zostanie zakończona:
 - Sprawdza możliwość wyłożenia serii i grup.
 - Jeśli to możliwe, dokłada płytki do innych grup i serii.

Tryby gry :

- Użytkownik przeciwko trzem graczom komputerowym
- Trzej gracze komputerowi przeciwko agentowi, tryb służy głównie do nauki modelu
- Użytkownik przeciwko modelowi, tryb służy głównie do testowania, czy model jest poprawnie nauczony.
- Użytkownik przeciwko dwóm graczom komputerowym i jednemu agentowi.

3.2 Wymagania нефункционалне

- **Kompatybilność :**
 - Gra musi działać na systemach Windows.
 - Rozgrywka wymaga klawiatury i myszki do sterowania.
- **Użyteczność :**
 - Interfejs użytkownika powinien być intuicyjny i zrozumiały dla nowych graczy.
 - Kolorystyka i oznaczenia płytek powinny odpowiadać klasycznym zasadom gry Rummikub.
- **Niezawodność :**
 - Gra nie zezwala graczom na wykonanie ruchów wbrew zasadom.

3.3 Opis narzędzi

W tej sekcji przedstawiono narzędzia, które zostały wykorzystane do zrealizowania projektu.

3.3.1 Unity

Środowisko gry zostało zaimplementowane z silnika Unity w najnowszej stabilnej wersji 2022.3.48f1. Unity jest wieloplatformowym silnikiem do tworzenia gier i aplikacji 3D jak i 2D stworzony przez Unity Technologies. Jest szeroko stosowany nie tylko do gier wideo, ale również do symulatorów, wizualizacji interaktywnych oraz aplikacji. Unity oferuje dużo możliwości, takie jak:

- Tworzenie modeli 2D i 3D
- Animacje i symulacja fizyki
- Renderowanie grafiki

3.3.2 C#

C# (C-Sharp) to nowoczesny, obiektowy język programowania opracowany przez firmę Microsoft. Łączy zalety języków C++ oraz Java, co czyni go uniwersalnym przy użytkowaniu. Może być wykorzystany do tworzenia gier wideo czy aplikacji mobilnych.

C# wyróżnia się następującymi cechami:

- Programowanie obiektowe.
- Zarządzanie pamięcią - posiadając funkcję garbage collector, nie trzeba martwić się o wycieki pamięci.
- Wszechstronność – umożliwia tworzenie aplikacji dla systemów Windows, macOS, Android, iOS oraz gier w środowisku Unity.

C# w projekcie został wykorzystany do zaimplementowania logiki gry, komunikacji z modelem oraz zarządzania interakcji między graczem a środowiskiem w Unity.

3.3.3 Python

Python to wszechstronny, wysokopoziomowy język programowania, głównie znany z prostoty i przejrzystości kodu. Najważniejsze cechy języka Python to:

- Bogata biblioteka standardowa – zawiera moduły do pracy z danymi, plikami, sieciami, obliczeniami naukowymi i innymi zadaniami.

- Przenośność i wszechstronność – Python działa na większości systemów operacyjnych, takich jak Windows, macOS czy Linux i jest wykorzystywany w aplikacjach desktopowych, webowych, a także w badaniach naukowych.
- Poprzez istnienie wielu zewnętrznych bibliotek, takich jak NumPy, TensorFlow, PyTorch, idealnie nadaje się do analizy danych.

Python w wersji 3.9 jest wykorzystany jako środowisko do implementacji modelu uczenia maszynowego, jest to konieczne w przypadku biblioteki ML-Agents.

3.3.4 ML-Agents

ML-Agents (Machine Learning Agents) to ,opracowany przez Unity Technologies, zestaw narzędzi do wykorzystywania technik sztucznej inteligencji i uczenia maszynowego w projektach w środowisku Unity. Narzędzie pozwala na implementację agentów, którzy mogą podejmować decyzje w zaimplementowanym środowisku.

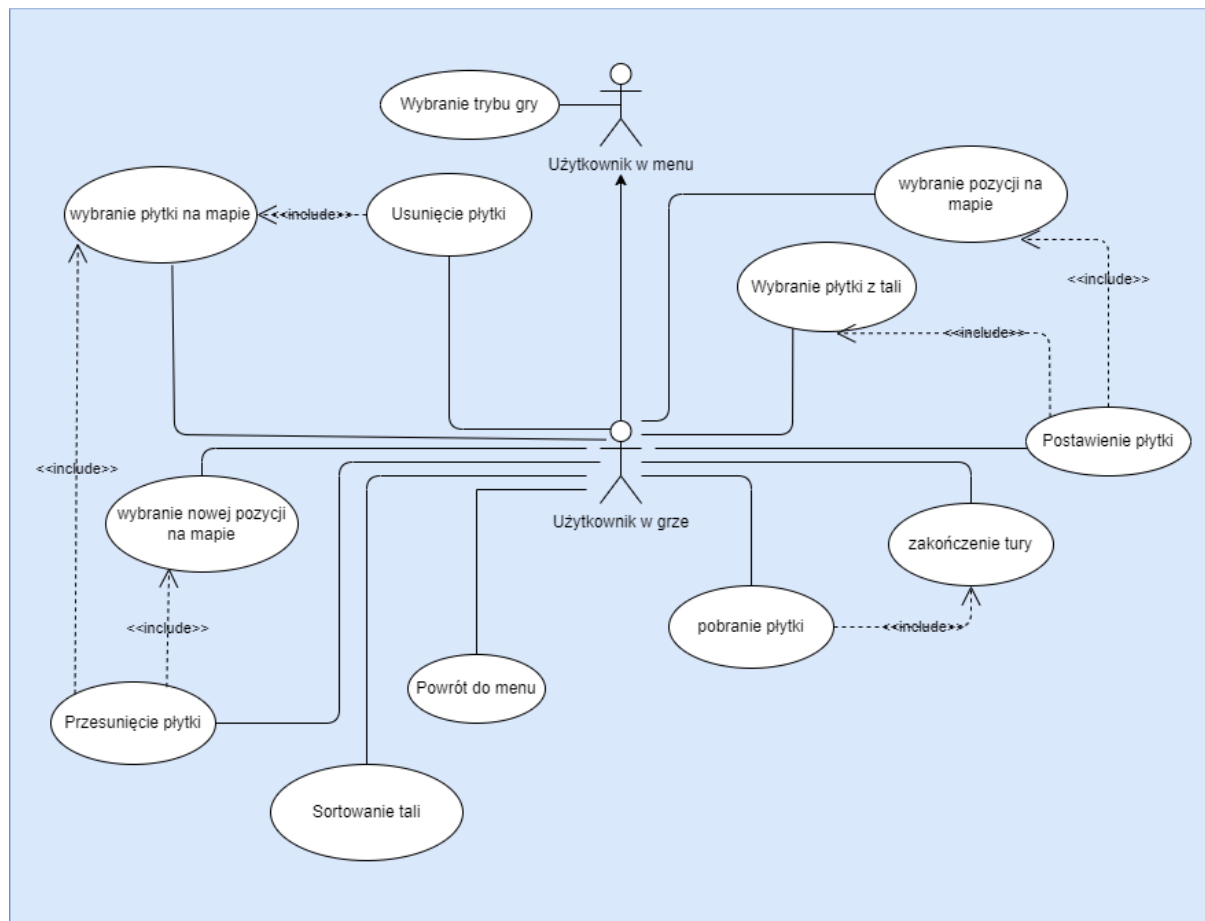
Kluczowymi cechami ML-Agents:

- Nauczanie wzmacniane - agenci uczą się na bazie dostawianych kar i nagród, jakie otrzymują.
- Narzędzie jest uniwersalne, można je zaimplementować do prostych jak i zaawansowanych gier, w których muszą podejmować decyzje.
- Agentów można zaimplementować do różnego rodzaju scenariuszy.

W tym projekcie, narzędzie, do poprawnej nauki wykorzystuje środowisko Python oraz unity. Unity jest miejscem implementacji akcji, obserwacji, nagród i kar dla modelu. Środowisko Python jest wykorzystywane do implementacji modelu oraz rozpoczynania nauki.

3.4 Przypadki użycia

Diagram przypadków użycia pokazuje możliwe funkcje dla gracza w menu oraz w grze.



Rysunek 3.1: Diagram przypadków użycia

Rozdział 4

Specyfikacja zewnętrzna

4.1 Wymagania sprzętowe i programowe

Minimalne wymagania zostały opisane w dokumentacji Unity [4], użytkownik powinien posiadać komputer z tymi bądź lepszymi komponentami:

- Windows 10 and Windows 11 tylko w wersji 64-bitowej
- CPU x64 z obsługą zestawu instrukcji SSE2.
- GPU kompatybilne z DX10, DX11 i DX12.

System był testowany na komputerze z komponentami:

- AMD Ryzen 9 7940HS w/ Radeon 780M Graphics 4.00 GHz
- 16,0 GB pamięci RAM
- System 64-bitowy Windows 11
- NVIDIA GeForce RTX 4050

4.2 Instalacja

Aby uruchomić grę w trybie edycyjnym, należy zainstalować poniższe oprogramowanie i biblioteki:

4.2.1 Unity

Do uruchomienia projektu wymagana jest instalacja Unity w wersji co najmniej 2022.3. Więcej informacji, na temat instalacji, znajduje się [5]. Aby pobrać i zainstalować Unity, należy:

1. Pobierz Unity Hub z oficjalnej strony: <https://unity.com/download>.
2. Zainstaluj Unity Hub i dodaj wersję edytora Unity 2022.3 lub nowszą.
3. Skonfiguruj środowisko projektu, upewniając się, że zainstalowane są wybrane moduły.

4.2.2 Python

Konieczne jest zainstalowanie Pythona w wersji 3.8 bądź nowsza, więcej informacji można znaleźć [1].

1. Należy pobrać instalator z odpowiednią wersją oficjalnej strony <https://www.python.org/downloads/>.
2. Uruchomić instalację.

4.2.3 Biblioteka ml-agents

Biblioteka ML-Agents Toolkit jest wymagana do wdrożenia komponentów sztucznej inteligencji w grze. Szczegółowa instrukcja instalacji znajduje się [3]. Konieczne jest zainstalowanie jej w środowisku Python jak i w Unity.

1. Przejdź do zakładki Okno (Window) w Unity
2. Wybierz Menedżer Pakietów (Package Manager)
3. W okienko wyszukiwania wpisz *ml-agents*
4. Zainstaluj wybraną wersję
5. W plikach projektu stwórz środowisko wirtualne Python przy użyciu *Wiersza poleceń*

```
python -m venv nazwa_folderu
```

bądź

```
py -m venv nazwa_folderu
```

6. Aktywuj środowisko przy użyciu:

```
nazwa_folderu\Scripts\activate
```

7. Zainstaluj w środowisku wirtualnym potrzebne biblioteki

```
py -m pip install --upgrade pip
pip install torch
```

aby zainstalować konkretną wersję biblioteki pytorch, należy wpisać komendę:

```
pip install torch==(wybrana wersja)
```

Ostatecznie należy wpisać komendę:

```
pip install mlagents
```

Jeśli ta komenda wywoła błąd, należy wpisać:

```
pip install mlagents --use-feature=2020-resolver
```

Aby zweryfikować, czy biblioteka została poprawnie zainstalowana, należy wpisać:

```
mlagents-learn --help
```

Jeśli biblioteka jest poprawnie zainstalowana, to polecenie wyświetli dostępne komendy w ramach biblioteki ml-agents.

4.3 Aktywacja

Po poprawnej instalacji i konfiguracji bibliotek oraz oprogramowania, można uruchomić aplikację.

Aby włączyć symulację aplikacji, należy wykonać następujące kroki:

1. Otwórz Unity Hub i wybierz projekt z listy w odpowiedniej wersji edytora Unity.
2. Kliknij przycisk **Open**, aby załadować projekt.
3. W edytorze Unity załaduj scenę **Menu**
4. Naciśnij przycisk **Play**, aby uruchomić grę w trybie symulacji.

Jeżeli gra została zbudowana jako aplikacja, aktywacja przebiega następująco:

1. Otwórz folder z plikami projektu (np. **Build/Release**).
2. Uruchom plik wykonywalny (na Windows jest to plik typu **.exe**).

Aby rozpocząć naukę sztucznej inteligencji (ML-Agents), należy wykonać kolejne polecenia:

1. Otwórz Unity Hub i wybierz projekt z listy w odpowiedniej wersji edytora Unity.
2. Kliknij przycisk **Open**, aby załadować projekt.
3. W edytorze Unity załaduj scenę **CP_AI_GAME**
4. W **Wierszu poleceń** przejdź do plików gry
5. Aktywuj środowisko wirtualne w Python
6. Wpisz poniższą komendę:

```
magents-learn --run-id=nazwa
```

7. Naciśnij przycisk **Play** w Unity.

4.4 Sposób obsługi

Sposób obsługi zawiera szczegółową instrukcję, jak korzystać z aplikacji z perspektywy użytkownika.

4.4.1 Zasady gry

Projekt gry Rummikub jest oparty na fizycznej grze planszowej, której zasady zostały opisane dokładnie w [2]. Gra wymaga od dwóch do czterech graczy. Gra podstawowa zawiera 106 płytek, w którym jest osiem serii o wartościach od jeden do trzynastie w czterech kolorach, oraz dwa jokery. Jokery to płytki specjalne, które mogą zastąpić każdą płytkę na mapie. Zadaniem każdego gracza jest jak najszybciej wyłożyć na stół wszystkie płytki, które zawiera w swojej tabliczce. W grze znajdują się dwa typy sekwencji płytek

- Grupy - jest to sekwencja płytek, w skład której wchodzi co najmniej trzy płytki o tej samej liczbie ale różnych kolorach. Joker może zastąpić płytkę w takiej sekwencji, grupa nie może zawierać więcej niż 4 płytki

- Serie - jest to sekwencja co najmniej trzech płytek w tym samym kolorze o różnych wartościach w kolejności od najmniejszej do największej. W takiej sekwencji może znajdować się maksymalnie 13 płytek, rozpoczynającą płytką jest zawsze o numerze jeden, a kończącą o numerze trzynaście.

Każdy gracz zaczyna rozgrywkę z czternastoma płytkami. Na początku rozgrywki gracze muszą wyłożyć sekwencje i grupy o łącznej wartości trzydziestu punktów i więcej. Każda płytka ma swoją wagę punktową, zależną od liczby wypisanej na płytce. Wartością wagową jokera jest liczba jaką zastępuje na planszy. Dopóki gracz nie wykona pierwszego ruchu, nie może manipulować żadnymi płytkami na mapie. Gracz również nie może manipulować żadnymi sekwencjami na mapie w trakcie swojego pierwszego wyłożenia. Po zakończeniu pierwszej tury, gracz może wykladać grupy i serie oraz wykonywać akcje, takie jak:

- dokładanie płytek na końcu i początku sekwencji na mapie
- rozdzielać jedną większą sekwencje na dwie mniejsze
- podmieniać jokery na mapie z płytkami ze swojego stojaka.
- przebudowywać grupy na serie oraz serie na grupy

Gracz musi w każdej turze wyłożyć co najmniej jedną płytkę, jeśli nie może tego zrobić, musi pobrać płytkę z banku, co kończy jego turę i uniemożliwia wykonanie ruchu na mapie. W grze obowiązuje limit czasu. Jeśli gracz nie zakończy swoich manipulacji przed upływem czasu, wszystkie jego ruchy należy cofnąć, a gracz dobiera płytkę z głównego banku. Gra może zakończyć się w dwóch momentach:

1. gdy jeden gracz wyłoży wszystkie płytki ze swojego stojaka
2. jeśli w głównym banku skończą się płytki a gracz, który ma swoją turę, chce dobrać kolejną.

Wynik gry zależy od płytek w stojaku gracza. Na końcu gry są podliczane wyniki na podstawie wag wartości płytek. Wagi jokerów są równe 30. Wygrywa ten, którego wynik jest najmniejszy.

4.4.2 Ekran aplikacji

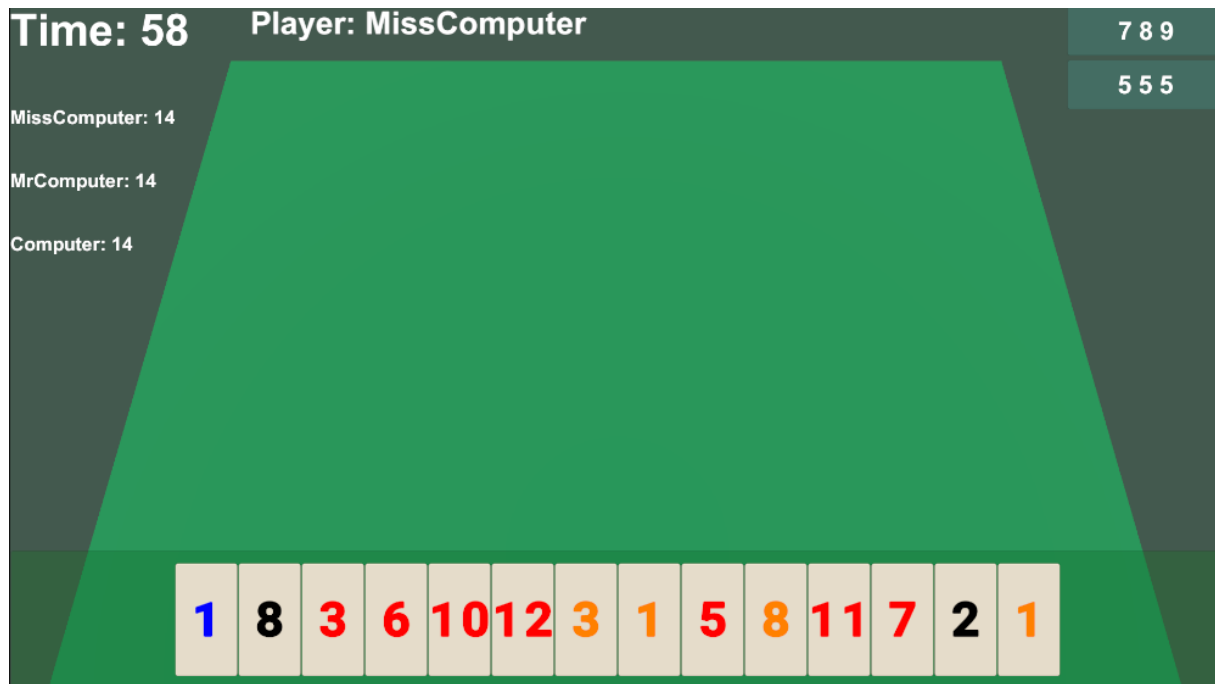
Rozgrywka rozpoczyna się widokiem menu głównego gry



Rysunek 4.1: Menu gry

Na ekranie widać cztery przyciski, każdy z nich wykonuje co innego:

- **Play with computer players** - jest to rozgrywka, w której biorą udział trzech gracze komputerowi z predefiniowanymi ruchami wraz z użytkownikiem. Tryb służy jedynie do potencjalnego zapoznania się z grą i jej mechanikami.
- **Play with AI and computer players** - tryb gry, w którym użytkownik gra wraz z dwoma graczami komputerowymi oraz jednym graczem komputerowym z algorytmami sztucznej inteligencji. Jest to docelowy tryb projektu.
- **AI vs computer players** - w tym trybie gry jest jeden gracz z algorytmami sztucznej inteligencji oraz trzech gracze komputerowi z predefiniowanymi ruchami. Ten tryb służy tylko do nauki modelu.
- **Play with AI** - ten tryb gry jest grą jeden na jeden, gdzie użytkownik gra przeciwko komputerowemu graczowi z uczeniem maszynowym. Ten tryb służy do walidacji nauki danego modelu.
- **Quit game** - przycisk służący do opuszczenia gry.



Rysunek 4.2: Ekran początku gry

4.4.3 Przebieg gry

Rozdział 5

Specyfikacja wewnętrzna

TODO

5.1 Biblioteka ml-agents

5.2 Klasa PlayerIA

5.3 Klasa GameController

5.4 Klasa Tile

Rozdział 6

Weryfikacja i walidacja

6.1 Testowanie

6.2 Wykryte błędy

Rozdział 7

Podsumowanie i wnioski

7.1 Napotkane problemy

7.2 Możliwy rozwój

Bibliografia

- [1] Python Software Foundation. *Beginners Guide to Python: Download*. 2024. URL: <https://wiki.python.org/moin/BeginnersGuide/Download> (term. wiz. 05.11.2024).
- [2] Rummikub. *Zasady gry Rummikub*. PDF dokument. `rummikub_rules.pdf`. 2019.
- [3] Unity Technologies. *ML-Agents Toolkit: Installation Guide*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Installation.md> (term. wiz. 11.12.2024).
- [4] Unity Technologies. *Unity System Requirements*. 2023. URL: <https://docs.unity3d.com/2022.3/Documentation/Manual/system-requirements.html> (term. wiz. 10.12.2024).
- [5] Unity Technologies. *Unity System Requirements*. 2023. URL: <https://docs.unity3d.com/550/Documentation/Manual/InstallingUnity.html> (term. wiz. 11.12.2024).

Dodatki

Spis skrótów i symboli

Źródła

Lista dodatkowych plików, uzupełniających tekst pracy

Spis rysunków

3.1	Diagram przypadków użycia	9
4.1	Menu gry	16
4.2	Ekran początku gry	17

Spis tabel